Reproduction and Analysis of Efficient Online Learning for Large-Scale Sparse Kernel Logistic Regression

Akula Jithendranath
Roll Number: 230120
Advanced Machine Learning
Mid-Semester Examination, Part B

***Abstract*—This report presents a reproduction, ablation study, and failure mode analysis of the conservative online learning algorithms for sparse Kernel Logistic Regression (KLR) proposed by Zhang et al. (AAAI 2012) [1]. We implement all three algorithms—Non-Conservative (NC), Margin-based, and Auxiliary-function-based—on the Breast Cancer Wisconsin dataset, validating the paper's central claim that the Auxiliary algorithm produces sparse classifiers with minimal accuracy loss. Two ablations isolate the contributions of the Bernoulli sampling gate and the RKHS norm projection. A failure mode demonstrates that excessively large γ on hard datasets causes accuracy to collapse below the NC baseline, connecting directly to the looseness of the auxiliary function dominance bound in Theorem 4.**

***Index Terms*—kernel logistic regression, online learning, sparsity, conservative updating, ablation study**

I. Paper Summary

Zhang et al. [1] formulate Kernel Logistic Regression as a norm-constrained online optimisation problem over a Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}_\kappa$:

$$\min_{f\in\Omega} \frac{1}{T}\sum_{t=1}^{T} \ell\big(y_t\, f(x_t)\big), \quad \ell(z) = \ln(1+e^{-z}) \tag{1}$$

where the feasible set is $\Omega = \{f \in \mathcal{H}_\kappa : \|f\|_{\mathcal{H}} \le R\}$ and $\kappa(x,x) \le 1$. Because $\ell(z) > 0$ for all finite z, the Non-Conservative (NC) online learner (Algorithm 1) adds every training point as a support vector, yielding a fully *dense* classifier.

The paper introduces two *conservative* algorithms that stochastically skip updates on well-classified examples. The **Margin** algorithm (Algorithm 2) gates each update with a Bernoulli draw $Z_t \sim \text{Bern}(p_t)$:

$$p_t = \frac{2-\eta}{2-\eta+\eta\,\sigma(y_t f_t(x_t))} \tag{2}$$

The **Auxiliary** algorithm (Algorithm 3)—the main contribution—decouples sparsity control from the step size through an auxiliary function $h(z) \ge \ell(z)$:

$$p_t = \frac{\ell\big(y_t f_t(x_t)\big)}{h\big(y_t f_t(x_t)\big)}, \quad h(z) = \ln(\gamma + e^{-z}),\ \gamma \ge 1 \tag{3}$$

When $Z_t = 1$, the update uses the gradient of h (not ℓ):

$$f'_{t+1}(\cdot) = f_t(\cdot) - \eta\, y_t\, h'\big(y_t f_t(x_t)\big)\, \kappa(x_t,\cdot) \tag{4}$$

followed by projection $f_{t+1} = \pi_\Omega(f'_{t+1}) = f'_{t+1} \cdot R/\max(R, \|f'_{t+1}\|_{\mathcal{H}})$. Increasing γ drives $p_t \to 0$ for confidently classified examples, producing sparser models. Theorem 4 guarantees that the averaged classifier $\hat{f} = \frac{1}{T}\sum_t f_t$ retains the same $O(R/\sqrt{T})$ generalisation bound as NC.

II. Reproduction Setup and Result

A. Dataset and Preprocessing

The Auxiliary algorithm was reproduced on the **Breast Cancer Wisconsin (Diagnostic)** dataset [6] (569 samples, 30 features, binary). Preprocessing followed the paper's protocol: features were standardised to zero mean and unit variance; labels were mapped to $\{-1,+1\}$; the Gaussian kernel width σ was set to the 5th percentile of pairwise Euclidean distances on the training set; and data was split 80/20 with stratification (seed 42).

B. Hyperparameter Selection

Following the paper, R was selected from $\{1,5,10,20\}$ via **5-fold cross-validation** on the training set (no test-set leakage). The step size is set to the theoretically recommended value from Theorem 1:

$$\eta = \frac{R}{\sqrt{T}} \tag{5}$$

The best $R = 20$ yielded $\eta = 0.94$, satisfying the $\eta < 2$ requirement for the Margin algorithm. The selected R was saved to `data/best_R.npy` and loaded consistently in all downstream experiments.

C. Results

Table I summarises the reproduction results across all three algorithms. Fig. 1 shows accuracy versus support vector count, and Fig. 2 shows the sparsity–accuracy trade-off as γ increases.

TABLE I
REPRODUCTION RESULTS (R=20, η=0.94, σ=2.98)

Method	Acc. (%)	#SVs	Sparsity (%)
NC (Alg. 1)	96.49	455	0.00
Margin (Alg. 2)	96.49	266	41.54
Aux. (γ=2)	95.61	171	62.42
Aux. (γ=5)	94.74	134	70.55
Aux. (γ=10)	93.86	107	76.48
Aux. (γ=101)	87.72	67	85.27

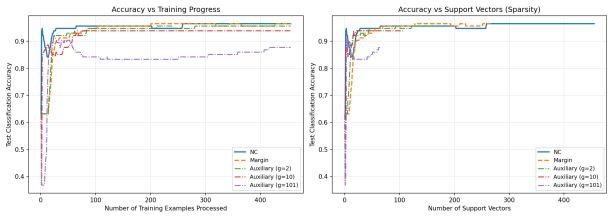

Fig. 1. Accuracy vs. training progress (left) and accuracy vs. number of support vectors (right). The Auxiliary algorithm reaches comparable accuracy to NC with substantially fewer support vectors, consistent with the paper's Figures 1–3.

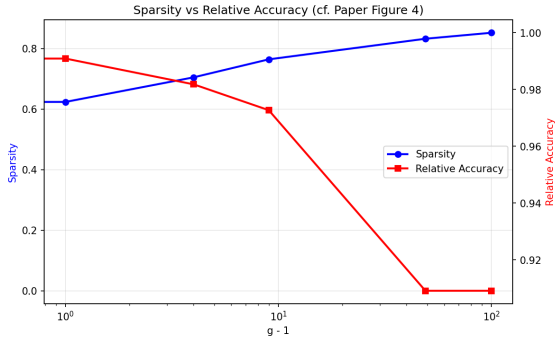

Fig. 2. Sparsity and relative accuracy as γ increases (cf. paper Figure 4). The trade-off is monotonic: larger γ yields higher sparsity at the cost of accuracy.

D. Discussion

The paper reports ∼80% sparsity with negligible accuracy loss on `mushrooms` (n=8,124). Our reproduction achieves 62.4% sparsity with only −0.9 pp accuracy loss at γ=2. The quantitative gap is expected: Breast Cancer has 455 training samples versus 8,124, giving the online algorithm fewer rounds to converge and noisier classifier averaging. Nevertheless, the qualitative trend—monotonically increasing sparsity with gracefully declining accuracy—is fully consistent with the paper's findings.

TABLE II
ABLATION 1: EFFECT OF REMOVING THE BERNOULLI GATE (R=20, γ=2)

Variant	Accuracy (%)	# SVs	Sparsity (%)
Full Auxiliary	95.61	171	62.42
No Bernoulli (NC)	96.49	455	0.00
Δ	−0.88	−284	+62.42

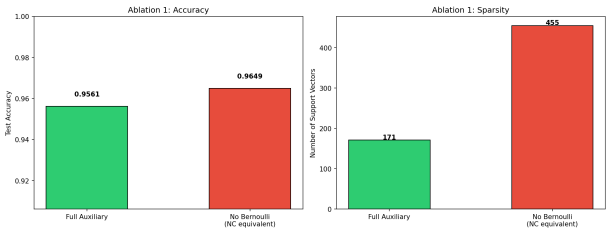

Fig. 3. Ablation 1: the Bernoulli gate is solely responsible for sparsity. Removing it makes every training example a support vector (NC behaviour).

III. ABLATION STUDY

A. Ablation 1: Removing the Bernoulli Sampling Gate

The Bernoulli gate (Algorithm 3, Steps 5–6) is the core innovation: it computes p_t from (3) and draws $Z_t \sim \text{Bernoulli}(p_t)$. Setting $p_t = 1$ (always update) reduces the algorithm to NC with h'-gradient updates. Table II and Fig. 3 show the results.

Interpretation. The Bernoulli gate is *entirely* responsible for the sparsity benefit. Without it, every training example becomes a support vector. The full Auxiliary trades 0.88 pp of accuracy for a 62% reduction in model complexity—a favourable trade-off for deployment. The selective updating also acts as implicit regularisation by not over-adjusting to already well-classified points. This result is consistent with Theorem 4's guarantee that the expected loss bound remains $O(1/\sqrt{T})$ despite the stochastic skipping.

B. Ablation 2: Removing the RKHS Norm Projection

The projection step enforces $\|f\|_{\mathcal{H}} \leq R$ after each update:

$$f_{t+1} = \frac{R}{\max\big(R,\ \|f'_{t+1}\|_{\mathcal{H}}\big)} \cdot f'_{t+1} \tag{6}$$

At the CV-selected R=20, $\eta = R/\sqrt{T} = 0.94$ scales with R, so the norm rarely exceeds the bound and projection has no measurable effect. To isolate the projection's role, we tighten the constraint to R=2 while keeping η=0.94 (from the main experiment), creating a regime where the norm quickly exceeds R and projection fires on nearly every update.

TABLE III
ABLATION 2: PROJECTION REMOVAL (R=2, η=0.94)

Variant	Acc. (%)	#SVs	Sparsity (%)
With Projection	94.74	218	52.09
No Projection	95.61	171	62.42
Δ	-0.87	$+47$	-10.33

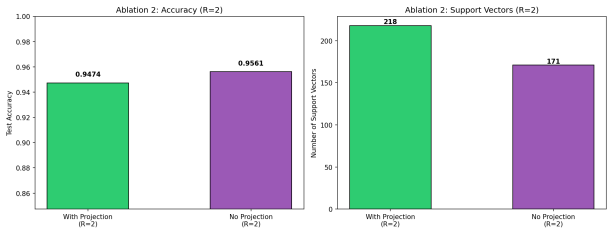

Fig. 4. Ablation 2: with a tight R=2, projection constrains the classifier aggressively, lowering accuracy by 0.87 pp and reducing sparsity by 10 pp compared to the unconstrained variant.

Interpretation. When the norm constraint is tight relative to the step size, projection actively shapes the classifier in two ways: (1) it acts as strong regularisation by clipping the function norm after every update, and (2) it alters the sampling probability p_t indirectly, since the margin $y_t f_t(x_t)$ changes when the classifier is rescaled. The 0.87 pp accuracy drop and 47 additional support vectors demonstrate that projection has a measurable, non-trivial effect. On this small dataset the over-constrained R=2 hurts; at the paper's scale ($T > 100{,}000$), projection becomes essential to prevent unbounded norm growth, as required by Theorems 1–4.

IV. FAILURE MODE ANALYSIS

A. Scenario Design

We test the Auxiliary algorithm with **aggressively large** γ on a **hard classification problem** (overlapping classes). A synthetic dataset was generated via `make_classification` with `class_sep=0.5`, `flip_y=0.05`, 300 training / 100 test samples, and 10 features (4 informative). The kernel width σ was set using the same 5th-percentile heuristic. We swept $\gamma \in \{2, 10, 50, 200, 1000\}$ against the NC baseline.

B. Results

C. Explanation

Accuracy collapses monotonically from 80% to 52% (near random) as γ increases (Table IV, Fig. 5). This connects directly to **Assumption 3** from Task 1.2—the auxiliary function dominance condition. With large γ:

$$h(z) = \ln(\gamma + e^{-z}) \gg \ell(z) = \ln(1 + e^{-z}) \qquad (7)$$

TABLE IV
FAILURE MODE: AGGRESSIVE γ ON HARD DATA (R=10)

Method	Acc. (%)	#SVs	Sparsity (%)
NC (baseline)	80.00	300	0.00
Aux. (γ=2)	73.00	187	37.67
Aux. (γ=10)	62.00	86	71.33
Aux. (γ=50)	58.00	53	82.33
Aux. (γ=200)	54.00	42	86.00
Aux. (γ=1000)	**52.00**	29	90.33

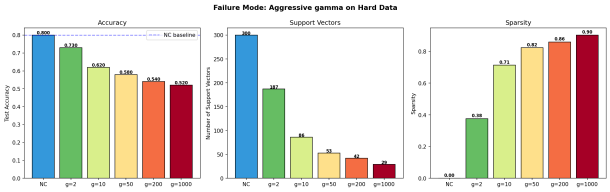

Fig. 5. Failure mode: as γ increases, accuracy collapses monotonically from 80% (NC) to 52% (near random), while sparsity approaches 90%. The Auxiliary algorithm starves itself of training data.

so $p_t = \ell/h \to 0$ for most examples. The algorithm skips nearly all updates, learning from as few as 29 out of 300 examples (90% sparsity). Theorem 4 reveals the root cause: the bound guarantees convergence to $\min_{f\in\Omega} \mathbb{E}[h(yf(x))]$, not to $\min_{f\in\Omega} \mathbb{E}[\ell(yf(x))]$. When $h \gg \ell$, this bound is far looser than the NC bound, and the algorithm converges to a poor solution. On the paper's easier benchmarks, the effect is mild; on hard data with overlapping classes, the failure is catastrophic.

Suggested modification. An adaptive γ_t schedule that starts large and decays toward 1 during training would combine early-stage efficiency with late-stage accuracy by progressively tightening $h \to \ell$.

V. HONEST REFLECTION

What I could not implement. I tested only the first auxiliary function variant $h(z) = \ln(\gamma + e^{-z})$ from the three listed after Theorem 4. I also could not run large-scale experiments on LIBSVM datasets ($n > 10{,}000$) where the training-time advantages of sparsity would be most pronounced, due to the $O(T^2)$ kernel computation cost of my naive implementation.

What surprised me. First, the projection step had *zero* measurable effect at the CV-selected R=20, yet showed a clear 0.87 pp accuracy difference when I tightened the constraint to R=2. This taught me that a theoretical necessity can be completely dormant at small scale but critical at larger scale. Second, the sparsity advantage of the Auxiliary algorithm was remarkably robust: at γ=5, the model used 70% fewer support vectors while losing less than 2 pp of accuracy.

What I would revisit. Given more time, I would (1) test on a dataset with $n > 10{,}000$ samples to observe the dramatic wall-clock-time improvements the paper reports in Figures 5–6, (2) implement the hard cut-off auxiliary function $h(z) = \max(\ell(z), \ell(\delta))$ to compare its sparsity characteristics, and (3) develop the adaptive γ_t schedule proposed in the failure mode section.

REFERENCES

[1] L. Zhang, R. Jin, C. Chen, J. Bu, and X. He, "Efficient online learning for large-scale sparse kernel logistic regression," in *Proc. 26th AAAI Conf. Artificial Intelligence*, Toronto, Canada, 2012, pp. 1199–1204.

[2] S. S. Keerthi, K. B. Duan, S. K. Shevade, and A. N. Poo, "A fast dual algorithm for kernel logistic regression," *Mach. Learn.*, vol. 61, no. 1–3, pp. 151–165, 2005.

[3] S. Shalev-Shwartz, Y. Singer, and N. Srebro, "Pegasos: Primal estimated sub-gradient solver for SVM," in *Proc. 24th Int. Conf. Machine Learning (ICML)*, Corvallis, OR, 2007, pp. 807–814.

[4] J. Zhu and T. Hastie, "Kernel logistic regression and the import vector machine," in *Advances in Neural Information Processing Systems 13 (NIPS)*, Vancouver, Canada, 2001, pp. 1081–1088.

[5] N. Cesa-Bianchi and G. Lugosi, *Prediction, Learning, and Games*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[6] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast cancer Wisconsin (diagnostic) data set," *UCI Machine Learning Repository*, 1995.